<u>Docker lab 1</u>

<u>Problem 1:</u>

- ➔ Run the container hello-world
- ➔ Check the container status
- ➔ Start the stopped container
- ➔ Remove the container
- ➔ Remove the image

<u>Problem 2:</u>

- ➔ Run container centos or ubuntu in an interactive mode
- ➔ Run the following command in the container “echo docker”
- ➔ Open a bash shell in the container and touch a file named hello-docker
- ➔ Stop the container and remove it. Write your comment about the file hello-docker
- ➔ Remove all stopped containers

<u>Problem 3:</u>

- ➔ Deploy a MySQL database called app-database. Use the mysql latest image, and use the -e flag to set MYSQL_ROOT_PASSWORD to P4sSw0rd0!. The container should run in the background.

<u>Problem 4:</u>

- ➔ Run the image Nginx
- ➔ Add html static files to the container and make sure they are accessible by web users

<u>Problem 5:</u>

- ➔ Create a dockerfile for nginx image with different html content and different nginx conf that listen to port 8080 instead of port 80 on the container
- ➔ Run a container from the new image

Problem 6:

- ➔ Create a dockerfile to containerize a simple (react OR python OR node) app
- ➔ Run a container from the new image

Problem 7:

- ➔ Create a dockerfile for ubuntu image which sleeps according to the given number in the docker command (hint: CMD override) or sleeps by default for 15 sec (if not overridden by the user)
- ➔ Run a container from the new image and test the 2 scenarios

Problem 8:

- ➔ create a dockerfile for the same app of Problem 6 in smaller size using multi staging
- ➔ Run a container from the new image

Problem 9:

- ➔ Push the images created in Problem 7&8 into your docker hub repo (hint: create an account in dockerhub and use docker login command in your terminal)